LEARNING MATERIAL

UNIT-IV

LOGIC CONCEPTS

Syllabus:

Logic concepts: First order logic. Inference in first order logic, propositional vs. first order inference, unification & lifts, Forward chaining, Backward chaining, Resolution, Learning from observation Inductive learning, Decision trees, Explanation based learning, Reinforcement Learning.

Learning Outcomes:

At the end of the unit, students will be able to

- Apply resolution and explanation-based learning techniques.
- differentiate Forward chaining and Backward chaining.

Module 1: First order logic

Module 2: Learning

1. FIRST ORDER LOGIC

- Sentences are added to a knowledge base using **TELL**, exactly as in propositional logic. Such sentences are called assertions.
- For example, we can assert that John is a king, Richard is a person, and all kings are persons:

  TELL(KB, King(John))

  TELL(KB, Person(Richard))

  TELL(KB, $\forall$ x: King(x) $\rightarrow$ Person(x)) .

  We can ask questions of the knowledge base using **ASK**.

  For example,

  ASK(KB, King(John))

  returns **true**.
- Questions asked with ASK are called **queries** or **goals**.
- Generally speaking, any GOAL query that is logically entailed by the knowledge base should be answered positively.
- For example, given the two preceding assertions, the query

ASK(KB, Person(John))

should also return true.

- We can ask quantified queries, such as

  ASK(KB, ∃ x: Person(x))

- The answer is true, but this is perhaps not as helpful as we would like.

2. PROPOSITIONAL VS. FIRST-ORDER INFERENCE

- Begin with some simple inference rules that can be applied to sentences with quantifiers to obtain sentences without quantifiers.
- These rules lead naturally to the idea that first-order inference can be done by converting the knowledge base to propositional logic and using propositional inference.

2.1 Inference rules for quantifiers

- Let us begin with universal quantifiers. Suppose our knowledge base contains the standard folkloric axiom stating that all **greedy kings are evil**:

  ∀ x: King(x) ∧ Greedy(x) → Evil(x)

- The above statement seems quite permissible to infer any of the following sentences:

  King(John) ∧ Greedy(John) → Evil(John)

  King(Richard) ∧ Greedy(Richard) → Evil(Richard)

  King(Father (John)) ∧ Greedy(Father (John)) → Evil(Father (John))

- The rule of **Universal Instantiation** (**UI**) says that we can infer any sentence obtained by substituting a **ground term** (a term without variables) for the variable.
- To write out the inference rule formally, we use the notion of **substitutions**.
- Let SUBST(θ,α) denote the result of applying the substitution θ to the sentence α. Then the rule is written

$$\frac{\forall v \ \ \alpha}{SUBST\ (\{v\ /\ g\}, \alpha)}$$

  for any variable **v** and ground term **g**.

- For example, the three sentences given earlier are obtained with the substitutions {x/John}, {x/Richard}, and {x/Father (John)}.
- In the rule for **Existential Instantiation** (EI), the variable is replaced by a single new constant symbol.

- The formal statement is as follows: for any sentence **α**, variable **v**, and constant symbol **k** that does not appear elsewhere in the knowledge base

$$\frac{\exists \text{ v } \alpha}{SUBST\ (\{v\ /\ k\}, \alpha)}$$

- For example, from the sentence

  x: Crown(x) ∧ OnHead(x, John)

  we can infer the sentence

  Crown(C1) ∧ OnHead(C1, John)

  As long as C1 does not appear elsewhere in the knowledge base.

- Basically, the existential sentence says there is some object satisfying a condition, and applying the existential instantiation rule just gives a name to that object.

2.2 Reduction to propositional inference

- Once we have rules for inferring nonquantified sentences from quantified sentences, it becomes possible to reduce first-order inference to propositional inference.
- The first idea is that, just as an existentially quantified sentence can be replaced by one instantiation, a universally quantified sentence can be replaced by the set of all possible instantiations.
- For example, suppose our knowledge base contains just the sentences

  ∀ x: King(x) ∧ Greedy(x) → Evil(x)

  King(John)

  Greedy(John)

  Brother (Richard, John) (4.1)

- Now we apply **UI** to the first sentence using all possible ground-term substitutions from the vocabulary of the knowledge base, in this case, {x/John} and {x/Richard}.
- We obtain

  King(John) ∧ Greedy(John) → Evil(John)

  King(Richard) ∧ Greedy(Richard) → Evil(Richard)

  and we discard the universally quantified sentence.

- Now, the knowledge base is essentially propositional if we view the ground atomic sentences King(John), Greedy(John), and so on - as proposition symbols. Therefore, we can apply any of the complete propositional algorithms to obtain conclusions such as Evil(John).

3. UNIFICATION AND LIFTING

- In the previous example, the first-order inference propositionalization approach is rather inefficient.
- For example, given the query Evil(x) and the knowledge base in Equation (4.1), it seems perverse to generate sentences such as King(Richard) ∧ Greedy(Richard) → Evil(Richard).
- Indeed, the inference of Evil(John) from the sentences

  ∀ x: King(x) ∧ Greedy(x) → Evil(x)

  King(John)

  Greedy(John)

  seems completely obvious to a human being. We now show how to make it completely obvious to a computer.

3.1 A first-order inference rule

- The inference that John is evil—that is, that {x/John} solves the query Evil(x) - works like this: to use the rule that greedy kings are evil, find some x such that x is a king and x is greedy, and then infer that this x is evil.
- More generally, if there is some substitution θ that makes each of the conjuncts of the premise of the implication identical to sentences already in the knowledge base, then we can assert the conclusion of the implication, after applying θ.
- In this case, the substitution θ ={x/John} achieves that aim.
- We can actually make the inference step do even more work. Suppose that instead of knowing Greedy(John), we know that everyone is greedy:

  ∀ y: Greedy(y) (4.2)
- Then we would still like to be able to conclude that Evil(John), because we know that John is a king (given) and John is greedy.
- In this case, applying the substitution {x/John, y/John} to the implication premises King(x) and Greedy(x) and the knowledge-base sentences King(John) and Greedy(y) will make them identical. Thus, we can infer the conclusion of the implication.
- This inference process can be captured as a single inference rule that we call Generalized Modus Ponens. For atomic sentences pi, Pi', and q, where there is a substitution θ such that SUBST(θ, Pi')= SUBST(θ, pi), for all i,

$$\frac{p_1',\ p_2',\ \ldots,\ p_n',\ (p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}.$$

- There are n+1 premises to this rule: the **n** atomic sentences Pi' and the one implication. The conclusion is the result of applying the substitution θ to the consequent q.
- For our example:

  $P1'$ is King(John) p1 is King(x)

  $P2'$ is Greedy(y) p2 is Greedy(x)

  θ is {x/John, y/John} q is Evil(x)

  SUBST(θ, q) is Evil(John)

- It is easy to show that Generalized Modus Ponens is a sound inference rule.
- First, we observe that, for any sentence p (whose variables are assumed to be universally quantified) and for any substitution θ,

  p |= SUBST(θ, p)

  holds by Universal Instantiation. It holds in particular for a θ that satisfies the conditions of the Generalized Modus Ponens rule. Thus, from $P1', \ldots, Pn'$ we can infer

  SUBST(θ, $P1'$) ∧ . . . ∧ SUBST(θ, Pn')

  and from the implication p1 ∧ . . . ∧ pn → q we can infer

  SUBST(θ, p1) ∧ . . . ∧ SUBST(θ, pn) → SUBST(θ, q)

- Now, θ in Generalized Modus Ponens is defined so that SUBST(θ, Pi')= SUBST(θ, pi), for all i; therefore the first of these two sentences matches the premise of the second exactly. Hence, SUBST(θ, q) follows by Modus Ponens.
- Generalized Modus Ponens is a lifted version of Modus Ponens—it raises Modus Ponens from ground (variable-free) propositional logic to first-order logic.
- The key advantage of lifted inference rules over propositionalization is that they make only those substitutions that are required to allow particular inferences to proceed.

3.2 Unification

- Lifted inference rules require finding substitutions that make different logical expressions look identical. This process is called unification and is a key component of all first-order inference algorithms.
- The UNIFY algorithm takes two sentences and returns a unifier for them if one exists:

  UNIFY(p, q)=θ where SUBST(θ, p)= SUBST(θ, q)

- Let us look at some examples of how UNIFY should behave. Suppose we have a query AskVars(Knows(John, x)): whom does John know? Answers to this query can be found by finding all sentences in the knowledge base that unify with Knows(John, x).
- Here are the results of unification with four different sentences that might be in the knowledge base:

  UNIFY(Knows(John, x), Knows(John, Jane)) = {x/Jane}

  UNIFY(Knows(John, x), Knows(y, Bill)) = {x/Bill, y/John}

  UNIFY(Knows(John, x), Knows(y, Mother (y))) = {y/John, x/Mother (John)}

  UNIFY(Knows(John, x), Knows(x, Elizabeth)) = fail
- The last unification fails because x cannot take on the values John and Elizabeth at the same time.
- Now, remember that Knows(x, Elizabeth) means “Everyone knows Elizabeth,” so we should be able to infer that John knows Elizabeth.
- The problem arises only because the two sentences happen to use the same variable name, x.
- The problem can be avoided by standardizing apart one of the two sentences being unified, which means renaming its variables to avoid name clashes.
- For example, we can rename x in Knows(x, Elizabeth) to x_{17} (a new variable name) without changing its meaning. Now the unification will work:

  UNIFY(Knows(John, x), Knows(x17, Elizabeth)) = {x/Elizabeth, x_{17}/John}
- An algorithm for computing most general unifiers as follows:

function UNIFY(x , y, θ) **returns** a substitution to make x and y identical

inputs: x , a variable, constant, list, or compound expression

y, a variable, constant, list, or compound expression

θ, the substitution built up so far (optional, defaults to empty)

if θ = failure **then return** failure

else if x = y **then return** θ

else if VARIABLE?(x) **then return** UNIFY-VAR(x , y, θ)

else if VARIABLE?(y) **then return** UNIFY-VAR(y, x , θ)

else if COMPOUND?(x) **and** COMPOUND?(y) **then**

return UNIFY(x.ARGS, y.ARGS, UNIFY(x.OP, y.OP, θ))

else if LIST?(x) **and** LIST?(y) **then**

return UNIFY(x.REST, y.REST, UNIFY(x.FIRST, y.FIRST, θ))

else return failure

function UNIFY-VAR(var, x , θ) **returns** a substitution

if {var/val} ∈ θ **then return** UNIFY(val , x , θ)

else if {x/val} ∈ θ **then return** UNIFY(var, val , θ)

else if OCCUR-CHECK?(var, x) **then return** failure

else return add {var/x } to θ

4. FORWARD CHAINING

- The idea is simple: start with the atomic sentences in the knowledge base and apply Modus Ponens in the forward direction, adding new atomic sentences, until no further inferences can be made.
- Definite clauses such as Situation → Response are especially useful for systems that make inferences in response to newly arrived information.
- Many systems can be defined this way, and forward chaining can be implemented very efficiently.

4.1 First-order definite clauses

- A definite clause either is atomic or is an implication whose predecessor is a conjunction of positive literals and whose consequent is a single positive literal.
- The following are first-order definite clauses:

  King(x) ∧ Greedy(x) → Evil(x)

  King(John)

  Greedy(y)

- Unlike propositional literals, first-order literals can include variables, in which case those variables are assumed to be universally quantified.
- Not every knowledge base can be converted into a set of definite clauses because of the single-positive-literal restriction, but many can.
- Consider the following problem:

The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.

- We will prove that West is a criminal. First, we will represent these facts as first-order definite clauses.
- "it is a crime for an American to sell weapons to hostile nations":

  American(x) ∧ Weapon(y) ∧ Sells(x, y, z) ∧ Hostile(z) → Criminal (x) (4.3)
- "Nono has some missiles." The sentence ∃x: Owns(Nono, x) ∧ Missile(x) is transformed into two definite clauses by Existential Instantiation, introducing a new constant M1:

  Owns(Nono, M1) (4.4)

  Missile(M1) (4.5)
- "All of its missiles were sold to it by Colonel West":

  Missile(x) ∧ Owns(Nono, x) → Sells(West, x, Nono) (4.6)
- We will also need to know that missiles are weapons:

  Missile(x) → Weapon(x) (4.7)

  and we must know that an enemy of America counts as "hostile":

  Enemy(x, America) → Hostile(x) (4.8)

  "West, who is American":

  American(West) (4.9)

  "The country Nono, an enemy of America":

  Enemy(Nono, America) (4.10)
- This knowledge base contains no function symbols and is therefore an instance of the class of Datalog knowledge bases.
- Datalog is a language that is restricted to first-order definite clauses with no function symbols.
- Datalog gets its name because it can represent the type of statements typically made in relational databases.

4.2 A simple forward-chaining algorithm

- Starting from the known facts, it triggers all the rules whose premises are satisfied, adding their conclusions to the known facts.
- The process repeats until the query is answered or no new facts are added.

Function FOL-FC-ASK(KB,α) **returns** a substitution or false

inputs: KB, the knowledge base, a set of first-order definite clauses

α, the query, an atomic sentence

local variables: new, the new sentences inferred on each iteration

repeat until new is empty

new = {}

for each rule **in** KB **do**

$(p_1 \wedge p_2 \wedge \ldots \wedge p_n \rightarrow q)$ = STANDARDIZE-VARIABLES(rule)

for each θ such that

SUBST(θ, $p_1 \wedge p_2 \wedge .. \wedge p_n$) = SUBST(θ, $P_1' \wedge P_2' \wedge \ldots \wedge P_n'$)

for some $P_1', P_2', \ldots, P_n'$ in KB

q' = SUBST(θ, q)

if q' does not unify with some sentence already in KB or new **then**

add q' to new

φ= UNIFY(q',α)

if φ is not fail **then return** φ

add new to KB

return false

5. BACKWARD CHAINING

- Backward Chaining work backward from the goal, chaining through rules to find known facts that support the proof.

5.1 A backward-chaining algorithm

- FOL-BC-ASK(KB, goal) will be proved if the knowledge base contains a clause of the form lhs → goal, where lhs (left-hand side) is a list of conjuncts.
- An atomic fact like American(West) is considered as a clause whose lhs is the empty list.

```
function FOL-BC-ASK(KB, query) returns a generator of substitutions
  return FOL-BC-OR(KB, query, { })

generator FOL-BC-OR(KB, goal, θ) yields a substitution
  for each rule (lhs ⇒ rhs) in FETCH-RULES-FOR-GOAL(KB, goal) do
    (lhs, rhs) ← STANDARDIZE-VARIABLES((lhs, rhs))
    for each θ' in FOL-BC-AND(KB, lhs, UNIFY(rhs, goal, θ)) do
      yield θ'

generator FOL-BC-AND(KB, goals, θ) yields a substitution
  if θ = failure then return
  else if LENGTH(goals) = 0 then yield θ
  else do
    first, rest ← FIRST(goals), REST(goals)
    for each θ' in FOL-BC-OR(KB, SUBST(θ, first), θ) do
      for each θ'' in FOL-BC-AND(KB, rest, θ') do
        yield θ''
```

5.2 Logic programming

- Logic programming is a technology that comes fairly close to embodying the declarative ideal, that systems should be constructed by expressing knowledge in a formal language and that problems should be solved by running inference processes on that knowledge.
- The ideal is summed up in equation **Algorithm = Logic + Control**.

- Prolog programs are sets of definite clauses written in a notation somewhat different from standard first-order logic.
- Prolog uses uppercase letters for variables and lowercase for constants - the opposite of our convention for logic.
- Commas separate conjuncts in a clause, and the clause is written "backwards" from what we are used to; instead of A ∧ B → C in Prolog we have C :- A, B.
- Here is a typical example:

  criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).

6. RESOLUTION

- A single inference rule resolution that yields a complete inference algorithm when coupled with any complete search algorithm.
- Now, we describe how to extend resolution to first-order logic.
- Resolution is obtain through Conjunctive normal form

6.1 Conjunctive normal form for first-order logic

- As in the propositional case, first-order resolution requires that sentences be in conjunctive normal form (CNF) - that is, a conjunction of clauses, where each clause is a disjunction of literals.
- Literals can contain variables, which are assumed to be universally quantified.
- For example, the sentence

  ∀x: American(x) ∧ Weapon(y) ∧ Sells(x, y, z) ∧ Hostile(z) → Criminal(x)

  becomes, in CNF,

  ~American(x) ∨ ~Weapon(y) ∨ ~Sells(x, y, z) ∨ ~Hostile(z) ∨ Criminal(x)
- Every sentence of first-order logic can be converted into an inferentially equivalent CNF sentence.
- We illustrate the procedure by translating the sentence "Everyone who loves all animals is loved by someone,"

  ∀x: [∀y: Animal(y) → Loves(x, y)] → [∃y: Loves(y, x)]
- The steps are as follows:

Eliminate implications:

∀x: [~∀y: ~Animal(y) ∨ Loves(x, y)] ∨ [∃y: Loves(y, x)]

Move ~ inwards: In addition to the usual rules for negated connectives, we need rules for negated quantifiers. Thus, we have

~∀x p becomes ∃x ~p

~∃x p becomes ∀x ~p

Our sentence goes through the following transformations:

$\forall x: [\exists y: \sim(\sim Animal(y) \vee Loves(x, y))] \vee [\exists y\ Loves(y, x)]$

$\forall x: [\exists y: \sim \sim Animal(y) \wedge \sim Loves(x, y)] \vee [\exists y\ Loves(y, x)]$

$\forall x: [\exists y: Animal(y) \wedge \sim Loves(x, y)] \vee [\exists y\ Loves(y, x)]$

Standardize variables: For sentences like $(\exists x: P(x)) \vee (\exists x: Q(x))$ which use the same variable name twice, change the name of one of the variables. This avoids confusion later when we drop the quantifiers. Thus, we have

$\forall x: [\exists y: Animal(y) \wedge \sim Loves(x, y)] \vee [\exists z: Loves(z, x)]$

Skolemize: Skolemization is the process of removing existential quantifiers by elimination.

In the simple case, it is just like the Existential Instantiation rule. Translate $\exists x: P(x)$ into $P(A)$, where A is a new constant.

Drop universal quantifiers: At this point, all remaining variables must be universally quantified. Moreover, the sentence is equivalent to one in which all the universal quantifiers have been moved to the left. We can therefore drop the universal quantifiers:

$[Animal(F(x)) \wedge \sim Loves(x, F(x))] \vee Loves(G(z), x)$

Distribute $\vee$ over $\wedge$:

$[Animal(F(x)) \vee Loves(G(z), x)] \wedge [\sim Loves(x, F(x)) \vee Loves(G(z), x)]$

Module 2: Learning

7. LEARNING

- An agent is learning if it improves its performance on future tasks after making observations about the world.

7.1 Forms of Learning

- Any component of an agent can be improved by learning from data. The improvements, and the techniques used to make them, depend on four major factors:

  1. Which component is to be improved?
  2. What prior knowledge the agent already has?
  3. What representation is used for the data and the component?
  4. What feedback is available to learn from?

7.1.1 Components to be learned / improved

- The components of different agents include:

  1. A direct mapping from conditions on the current state to actions.

2. A means to infer relevant properties of the world from the percept sequence.
3. Information about the way the world evolves and about the results of possible actions the agent can take.
4. Utility information indicating the desirability of world states.
5. Action-value information indicating the desirability of actions.
6. Goals that describe classes of states whose achievement maximizes the agent's utility.

7.1.2 Representation and prior knowledge

- Several examples of representations for agent components: propositional and first-order logical sentences for the components in a logical agent; Bayesian networks for the inferential components of a decision-theoretic agent, and so on.
- Effective learning algorithms have been devised for all of these representations.
- There is another way to look at the various types of learning. We say that learning a (possibly incorrect) general function or rule from specific input–output pairs is called inductive learning.

7.1.3 Feedback to learn from

- There are three types of feedback that determine the three main types of learning:

  1. Unsupervised learning
  2. Reinforcement learning
  3. Supervised learning

7.1.3.1 Unsupervised learning

- In unsupervised learning the agent learns patterns in the input even though no explicit feedback is supplied.
- The most common unsupervised learning task is clustering: detecting potentially useful clusters of input examples.
- For example, a taxi agent might gradually develop a concept of "good traffic days" and "bad traffic days" without ever being given labeled examples of each by a teacher.

7.1.3.2 Reinforcement learning

- In reinforcement learning the agent learns from a series of reinforcements i.e. rewards or punishments.
- For example, the lack of a tip at the end of the journey gives the taxi agent an indication that it did something wrong.
- The two points for a win at the end of a chess game tells the agent it did something right. It is up to the agent to decide which of the actions prior to the reinforcement were most responsible for it.

7.1.3.3 Supervised learning

- In supervised learning the agent observes some example input–output pairs and learns a function that maps from input to output.
- In component 1, the inputs are percepts and the output are provided by a teacher who says “Brake!” or “Turn left.”
- In component 2, the inputs are camera images and the outputs again come from a teacher who says “that’s a bus.”
- In component 3, the theory of braking is a function from states and braking actions to stopping distance in feet.
- In this case the output value is available directly from the agent’s percepts (after the fact); the environment is the teacher.

7.1.3.4 Semi-supervised learning

- In semi-supervised learning we are given a few labeled examples and must make what we can of a large collection of unlabeled examples.
- Even the labels themselves may not be the oracular truths that we hope for.
- Imagine that you are trying to build a system to guess a person’s age from a photo. You gather some labeled examples by snapping pictures of people and asking their age. That’s supervised learning.
- But in reality some of the people lied about their age. It’s not just that there is random noise in the data; rather the inaccuracies are systematic, and to uncover them is an unsupervised learning problem involving images, self-reported ages, and true (unknown) ages.
- Thus, both noise and lack of labels create a continuum between supervised and unsupervised learning.

7.2 Inductive Learning

- An algorithm for deterministic supervised learning is given as the correct value of the unknown function for particular inputs, and must try to recover the unknown function or something close to it provided by the feedback.
- For an example (x,f(x)) is a pair, where **x** is the input and **f(x)** is the output of the function applied to **x**. The task of pure inductive inference (or induction) is this: given a collection of examples of **f**, return a function **h** that approximates **f**. The function **h** is called a hypothesis.
- Here learning is difficult from conceptual point of view, it is not easy to tell whether any particular hypothesis (h) is good approximation of **f**.
- A good hypothesis will generalize well – that is, will predict unseen examples correctly. This is the fundamental problem of induction.

8. DECISION TREES

- Decision tree induction is one of the simplest and yet most successful forms of machine learning. We first describe the representation the hypothesis space and then show how to learn a good hypothesis.

8.1 The decision tree representation / decision tree as performance elements

- A decision tree represents a function that takes as input a vector of attribute values and returns a "decision" - a single output value.
- The input and output values can be discrete or continuous.
- A decision tree reaches its decision by performing a sequence of tests. Each internal node in the tree corresponds to a test of the value of one of the input attributes, A_i, and the branches from the node are labeled with the possible values of the attribute, $Ai = v_{ik}$.
- Each leaf node in the tree specifies a value to be returned by the function.
- As an example, we will build a decision tree to decide whether to wait for a table at a restaurant. The aim here is to learn a definition for the goal predicate **WillWait**.
- First we list the attributes that we will consider as part of the input:

  1. Alternate: whether there is a suitable alternative restaurant nearby.
  2. Bar: whether the restaurant has a comfortable bar area to wait in.
  3. Fri/Sat: true on Fridays and Saturdays.
  4. Hungry: whether we are hungry.
  5. Patrons: how many people are in the restaurant (values are None, Some, and Full).
  6. Price: the restaurant's price range ($, $$, $$$).
  7. Raining: whether it is raining outside.
  8. Reservation: whether we made a reservation.
  9. Type: the kind of restaurant (French, Italian, Thai, or burger).
  10. WaitEstimate: the wait estimated by the host (0–10 min, 10–30, 30–60, or >60).

- The decision tree for above attributes is shown in Figure 1.

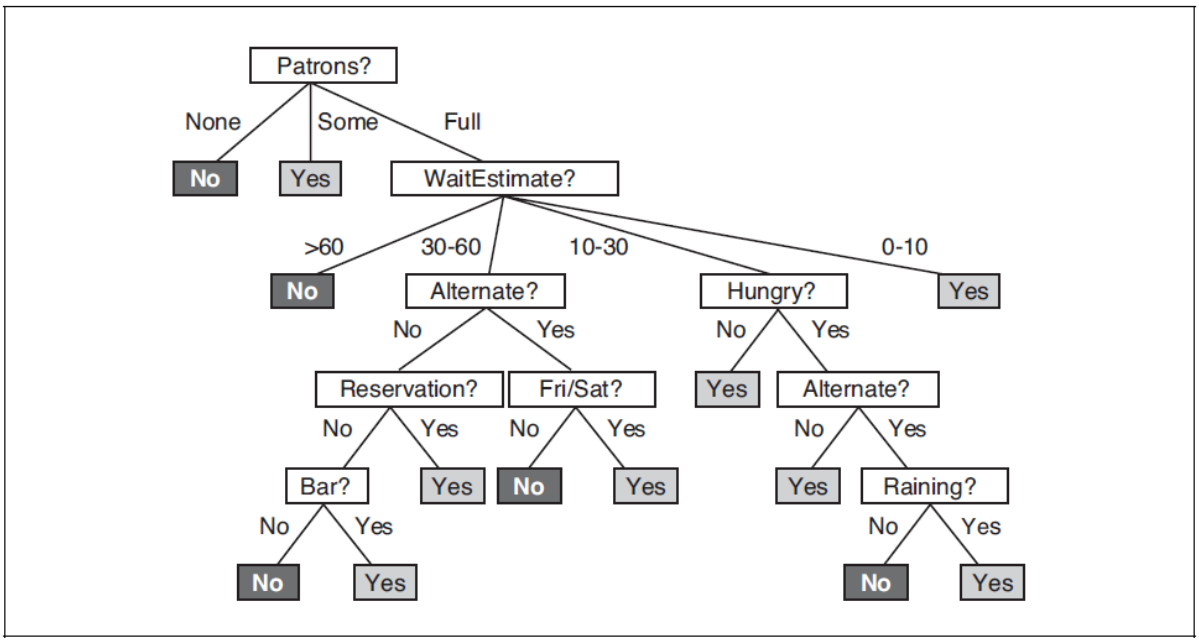


Figure 1: A decision tree for deciding whether to wait for a table.

8.2 Expressiveness of decision trees

- A Boolean decision tree is logically equivalent to the assertion that the goal attribute is true if and only if the input attributes satisfy one of the paths leading to a leaf with value true. Writing this out in propositional logic, we have

  Goal ⇔ (Path1 ∨ Path2 ∨ · · ·)

  where each Path is a conjunction of attribute-value tests required to follow that path. Thus, the whole expression is equivalent to disjunctive normal form, which means that any function in propositional logic can be expressed as a decision tree.

- As an example, the rightmost path in Figure 1 is

  Path = (Patrons = Full ∧ WaitEstimate = 0–10)

- For a wide variety of problems, the decision tree format yields a nice, concise result. But some functions cannot be represented concisely.

- For example, the majority function, which returns true if and only if more than half of the inputs are true, requires an exponentially large decision tree. In other words, decision trees are good for some kinds of functions and bad for others.

8.3 Inducing decision trees from examples

- An example for a Boolean decision tree consists of an (x, y) pair, where x is a vector of values for the input attributes, and y is a single Boolean output value.

- A training set of 12 examples is shown in Figure 2.

- The positive examples are the ones in which the goal WillWait is true (x1, x3, . . .); the negative examples are the ones in which it is false (x2, x5, . . .).

Example	Input Attributes										Goal
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
$\mathbf{x}_1$	*Yes*	*No*	*No*	*Yes*	*Some*	*$$$*	*No*	*Yes*	*French*	*0–10*	$y_1 = Yes$
$\mathbf{x}_2$	*Yes*	*No*	*No*	*Yes*	*Full*	*$*	*No*	*No*	*Thai*	*30–60*	$y_2 = No$
$\mathbf{x}_3$	*No*	*Yes*	*No*	*No*	*Some*	*$*	*No*	*No*	*Burger*	*0–10*	$y_3 = Yes$
$\mathbf{x}_4$	*Yes*	*No*	*Yes*	*Yes*	*Full*	*$*	*Yes*	*No*	*Thai*	*10–30*	$y_4 = Yes$
$\mathbf{x}_5$	*Yes*	*No*	*Yes*	*No*	*Full*	*$$$*	*No*	*Yes*	*French*	*>60*	$y_5 = No$
$\mathbf{x}_6$	*No*	*Yes*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Italian*	*0–10*	$y_6 = Yes$
$\mathbf{x}_7$	*No*	*Yes*	*No*	*No*	*None*	*$*	*Yes*	*No*	*Burger*	*0–10*	$y_7 = No$
$\mathbf{x}_8$	*No*	*No*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Thai*	*0–10*	$y_8 = Yes$
$\mathbf{x}_9$	*No*	*Yes*	*Yes*	*No*	*Full*	*$*	*Yes*	*No*	*Burger*	*>60*	$y_9 = No$
$\mathbf{x}_{10}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$$$*	*No*	*Yes*	*Italian*	*10–30*	$y_{10} = No$
$\mathbf{x}_{11}$	*No*	*No*	*No*	*No*	*None*	*$*	*No*	*No*	*Thai*	*0–10*	$y_{11} = No$
$\mathbf{x}_{12}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$*	*No*	*No*	*Burger*	*30–60*	$y_{12} = Yes$

Figure 2: Examples for the restaurant domain

The DECISION-TREE-LEARNING algorithm adopts a greedy divide-and-conquer strategy: always test the most important attribute first. This test divides the problem up into smaller subproblems that can then be solved recursively.

- Figure 3(a) shows that Type is a poor attribute, because it leaves us with four possible outcomes, each of which has the same number of positive as negative examples.
- On the other hand, in (b) we see that Patrons is a fairly important attribute, because if the value is None or Some, then we are left with example sets for which we can answer definitively (No and Yes, respectively).

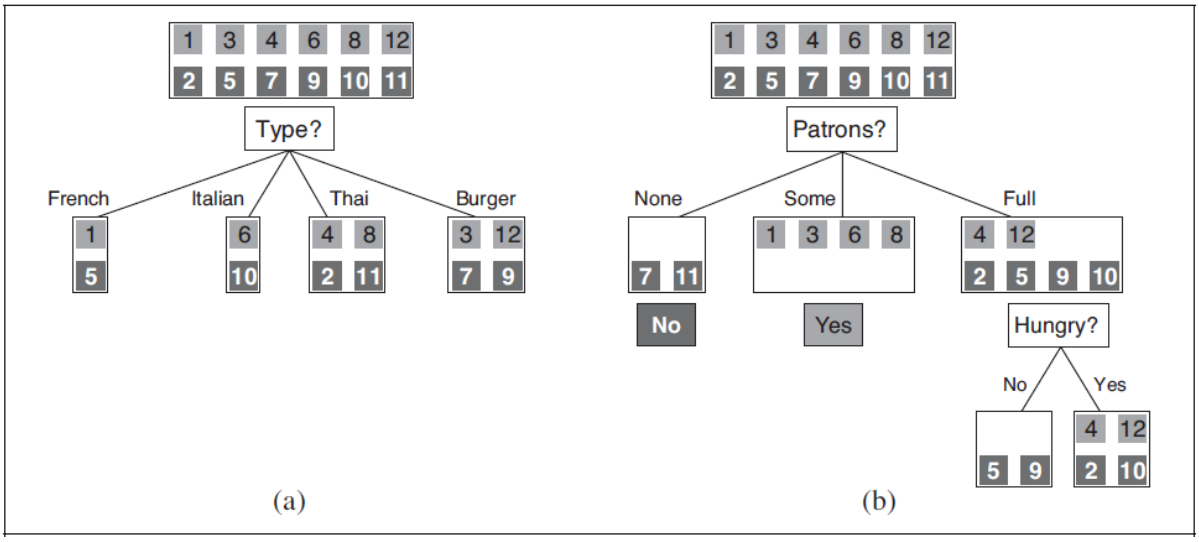


Figure 3: Splitting the examples by testing on attributes. At each node we show the positive (light boxes) and negative (dark boxes) examples remaining. (a) Splitting on Type brings us no nearer to distinguishing between positive and negative examples. (b) Splitting on Patrons does a good job of separating positive and negative examples. After splitting on Patrons, Hungry is a fairly good second test.

9. EXPLANATION BASED LEARNING

- Explanation-based learning is a method for extracting general rules from individual observations.
- For an example, consider the problem of differentiating and simplifying algebraic expressions. If we differentiate an expression such as X^2 with respect to X, we obtain 2X.
- In a logical reasoning system, the goal might be expressed as ASK(Derivative(X^2,X)=d, KB), with solution d = 2X.
- The technique of memoization has long been used in computer science to speed up programs by saving the results of computation.
- The basic idea of memo functions is to accumulate a database of input–output pairs; when the function is called, it first checks the database to see whether it can avoid solving the problem from scratch.
- Explanation-based learning takes this a good deal further, by creating general rules that cover an entire class of cases.
- In the case of differentiation, memoization would remember that the derivative of X^2 with respect to X is 2X, but would leave the agent to calculate the derivative of Z^2 with respect to Z from scratch.
- We would like to be able to extract the general rule that for any arithmetic unknown u, the derivative of u^2 with respect to u is 2u.
- In logical terms, this is expressed by the rule

  ArithmeticUnknown(u) ⇒ Derivative(u^2, u) = 2u.
- If the knowledge base contains such a rule, then any new case that is an instance of this rule can be solved immediately.

9.1 Extracting general rules from examples

- The basic idea behind explanation based learning is first to construct an explanation of the observation using prior knowledge, and then to establish a definition of the class of cases for which the same explanation structure can be used.
- This definition provides the basis for a rule covering all of the cases in the class.
- The “explanation” can be a logical proof, but more generally it can be any reasoning or problem-solving process whose steps are well defined.
- The key is to be able to identify the necessary conditions for those same steps to apply to another case.
- Suppose our problem is to simplify 1 × (0 + X). The knowledge base includes the following rules:

  Rewrite(u, v) ∧ Simplify(v, w) ⇒ Simplify(u, w)

Primitive(u) ⇒ Simplify(u, u)

ArithmeticUnknown(u) ⇒ Primitive(u)

Number(u) ⇒ Primitive(u)

Rewrite(1 × u, u)

Rewrite(0 + u, u)

- The proof that the answer is X is shown in the top half of Figure 4.
- In the figure 4, the first tree shows the proof for the original problem instance, from which we can derive

  ArithmeticUnknown(z) ⇒ Simplify(1 × (0 + z), z) .
- The second tree shows the proof for a problem instance with all constants replaced by variables, from which we can derive a variety of other rules.

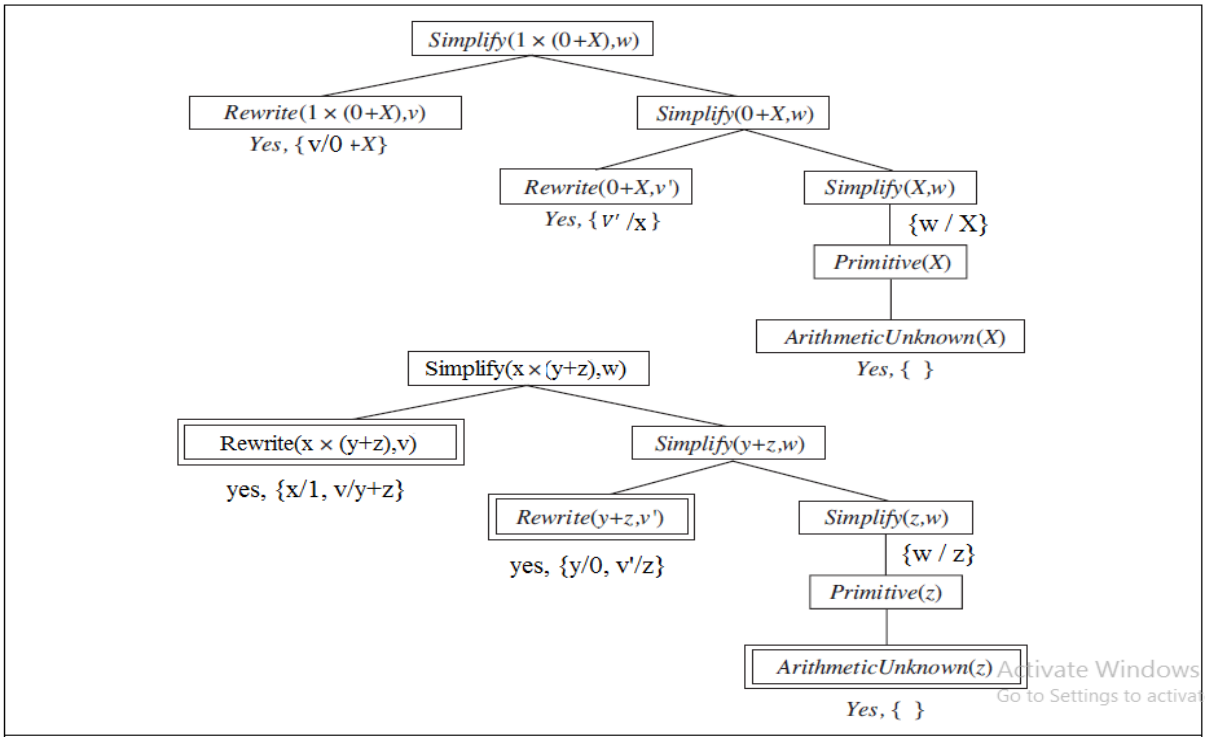


Figure 4: Proof trees for the simplification problem.

- The EBL method actually constructs two proof trees simultaneously. The second proof tree uses a variabilized goal in which the constants from the original goal are replaced by variables.
- As the original proof proceeds, the variabilized proof proceeds in step, using exactly the same rule applications. This could cause some of the variables to become instantiated.

- For example, in order to use the rule Rewrite(1×u, u), the variable x in the subgoal Rewrite(x×(y+z), v) must be bound to 1. Similarly, y must be bound to 0 in the subgoal Rewrite(y + z, v') in order to use the rule Rewrite(0 + u, u).
- The basic EBL process works as follows:
    1. Given an example, construct a proof that the goal predicate applies to the example using the available background knowledge.
    2. In parallel, construct a generalized proof tree for the variabilized goal using the same inference steps as in the original proof.
    3. Construct a new rule whose left-hand side consists of the leaves of the proof tree and whose right-hand side is the variabilized goal (after applying the necessary bindings from the generalized proof).
    4. Drop any conditions from the left-hand side that are true regardless of the values of the variables in the goal.

10. REINFORCEMENT LEARNING

- Here, we will study how agents can learn *what to do* in the absence of labeled examples of what to do.
- Consider, for example, the problem of learning to play chess. A supervised learning agent needs to be told the correct move for each position it encounters, but such feedback is rarely available.
- In the absence of feedback from a teacher, an agent can learn a transition model for its own moves and can perhaps learn to predict the opponent's moves, but *without some feedback about what is good and what is bad, the agent will have no grounds for deciding which move to make.*
- The agent needs to know that something good has happened when it (accidentally) checkmates the opponent, and that something bad has happened when it is checkmated or vice versa, if the game is loss. This kind of feedback is called a **reward**, or **reinforcement**.
- In games like chess, the reinforcement is received only at the end of the game. In other environments, the rewards come more frequently.
- Imagine playing a new game whose rules you don't know; after a hundred or so moves, your opponent announces, "You lose." This is reinforcement learning in a nutshell.
- In many complex domains, reinforcement learning is the only feasible way to train a program to perform at high levels.

- For example, in game playing, it is very hard for a human to provide accurate and consistent evaluations of large numbers of positions, which would be needed to train an evaluation function directly from examples.
- Instead, the program can be told when it has won or lost, and it can use this information to learn an evaluation function that gives reasonably accurate estimates of the probability of winning from any given position.
- Reinforcement learning might be considered to cover all of AI: an agent is placed in an environment and must learn to behave successfully therein.

10.1 Sequential Decision Problems

- Suppose that an agent is situated in the 4×3 environment shown in Figure 5(a). Beginning in the start state, it must choose an action at each time step.
- The interaction with the environment terminates when the agent reaches one of the goal states, marked +1 or –1.
- In the 4×3 environment, the actions in every state are *Up*, *Down*, *Left*, and *Right*.
- Assume that the environment is fully observable, so that the agent always knows where it is.

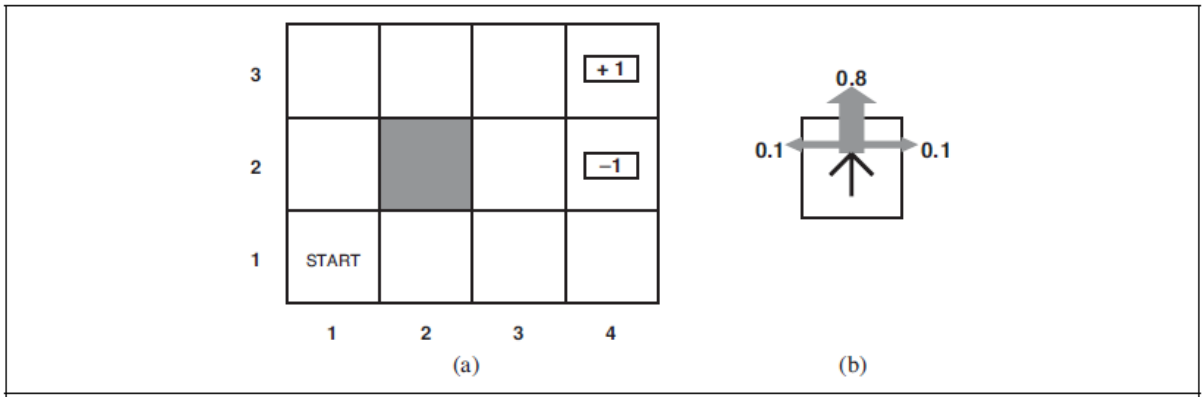
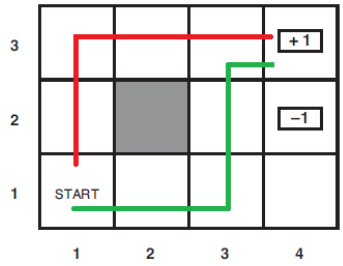


Figure 5: (a) A simple 4×3 environment that presents the agent with a sequential decision problem. (b) Illustration of the transition model of the environment: the "intended" outcome occurs with probability 0.8, but with probability 0.2 the agent moves at right angles to the intended direction. A collision with a wall results in no movement. The two terminal states have reward +1 and –1, respectively, and all other states have a reward of –0.04.

- If the environment were deterministic, a solution would be easy: [*Up, Up, Right, Right, Right*].
- Unfortunately, the environment won't always go along with this solution, because the actions are unreliable.
- The particular model of stochastic motion that we adopt is illustrated in Figure 5(b). Each action achieves the intended effect with probability 0.8, but the rest of the time, the action moves the agent at right angles to the intended direction. Furthermore, if the agent bumps into a wall, it stays in the same square.

- For example, from the start square (1, 1), the action *Up* moves the agent to (1, 2) with probability 0.8, but with probability 0.1, it moves right to (2, 1), and with probability 0.1, it moves left, bumps into the wall, and stays in (1, 1).



- In the above environment, the sequence [Up, Up, Right, Right, Right] goes up around the barrier and reaches the goal state at (4, 3) with probability $0.8^5 = 0.32768$. There is also a small chance of accidentally reaching the goal by going the other way around the sequence [Right, Right, Up, Up, Right] with probability $0.1^4 \times 0.8 = 0.00008$, for a grand total of 0.32768 + 0.00008 →0.32776.
- Here transition model describes the outcome of each action in each state. Here, the outcome is stochastic, so we write P(s' | s, a) to denote the probability of reaching state s' if action ***a*** is done in state ***s***.
- In the above example, the reward is −0.04 in all states except the terminal states (which have rewards +1 and –1).
- The utility of an environment history is just (for now) the *sum* of the rewards received.
- For example, if the agent reaches the +1 state after 10 steps, its total utility will be 0.6. The negative reward of –0.04 gives the agent an incentive to reach (4, 3) quickly, so our environment is a stochastic generalization of the search problems.
- What does a solution to the problem look like? We have seen that any fixed action sequence won't solve the problem, because the agent might end up in a state other than the goal.
- So, a solution must specify what the agent should do for any state that the agent might reach. A solution of this kind is called a **policy**. It is traditional to denote a policy by π, and π(s) is the action recommended by the policy π for state s.
- If the agent has a complete policy, the agent will always know what to do next.

11. PASSIVE REINFORCEMENT LEARNING

- To keep things simple, passive learning agent using a state-based representation in a fully observable environment.
- In passive learning, the agent's policy π is fixed: in state s, it always executes the action π(s). Its goal is simply to learn how good the policy is? i.e., to learn the utility function $U^{\pi}(s)$.
- Figure 6 shows a policy for that world and the corresponding utilities.

- The main difference is that the passive learning agent does not know the **transition model** P(s' | s, a), which specifies the probability of reaching state **s'** from state **s** after doing action **a**; nor does it know the **reward function** R(s), which specifies the reward for each state.

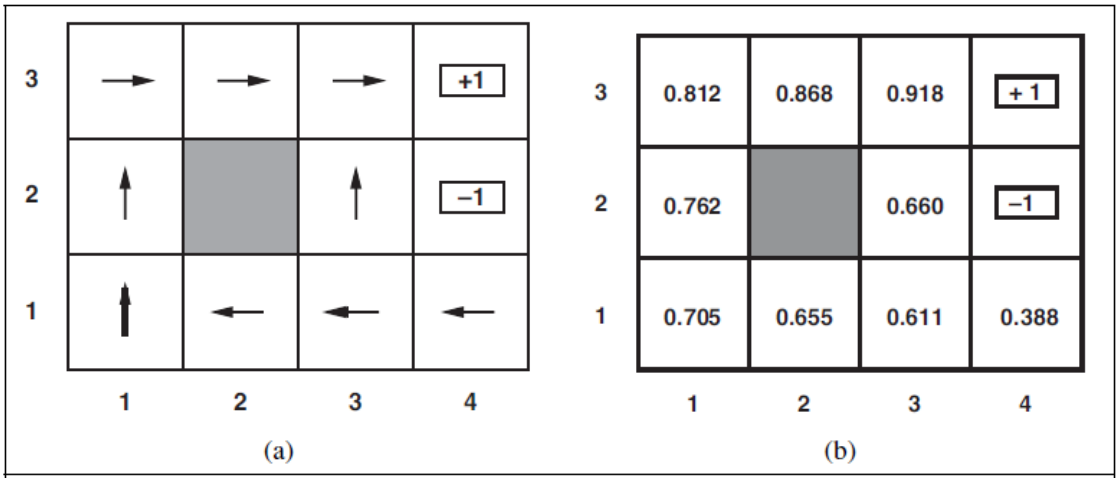


Figure 6: (a) A policy π for the 4×3 world; this policy happens to be optimal with rewards of R(s) = − 0.04 in the non-terminal states and no discounting. (b) The utilities of the states in the 4×3 world, given policy π.

- The agent executes a set of **trials** in the environment using its policy π.
- In each trial, the agent starts in state (1,1) and experiences a sequence of state transitions until it reaches one of the terminal states, (4,2) or (4,3). Its percepts supply both the current state and the reward received in that state.
- Typical trials might look like this:

(1, 1)-.04→(1, 2)-.04→(1, 3)-.04→(1, 2)-.04→(1, 3)-.04→(2, 3)-.04→(3, 3)-.04→(4, 3)+1

(1, 1)-.04→(1, 2)-.04→(1, 3)-.04→(2, 3)-.04→(3, 3)-.04→(3, 2)-.04→(3, 3)-.04→(4, 3)+1

(1, 1)-.04→(2, 1)-.04→(3, 1)-.04→(3, 2)-.04→(4, 2)-1 .

- Here each state percept is subscripted with the reward received. The object is to use the information about rewards to learn the expected utility $U^{\pi}(s)$ associated with each non-terminal state **s**.
- The utility is defined to be the expected sum of (discounted) rewards obtained if policy π is followed:

$$U^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t)\right] \qquad (4.11)$$

- Where R(s) is the reward for a state **s**, S_t (a random variable) is the state reached at time **t** when executing policy π, and S_0 = s.
- We will include a discount factor γ in all of our equations, but for the 4×3 world we will set γ =1.

12. DIRECT UTILITY ESTIMATION

- The idea is that the utility of a state is the expected total reward from that state onward, and each trial provides a *sample* of this quantity for each state visited.
- For example, the first trial in the set of three given earlier provides a sample total reward of 0.72 for state (1, 1), two samples of 0.76 and 0.84 for (1, 2), two samples of 0.80 and 0.88 for (1, 3), and so on.
- At the end of each sequence, the algorithm calculates the observed reward-to-go for each state and updates the estimated utility for that state accordingly, just by keeping a running average for each state in a table.
- In the limit of infinitely many trials, the sample average will converge to the true expectation in Equation (4.11).
- Direct utility estimation is just an instance of supervised learning where each example has the state as input and the observed reward-to-go as output.
- Direct utility estimation succeeds in reducing the reinforcement learning problem to an inductive learning problem, about which much is known.
- The utility of each state equals its own reward plus the expected utility of its successor states. That is, the utility values obey the Bellman equations for a fixed policy.

$$U^{\pi}(s) = R(s) + \gamma \sum_{s'} P\big(s' \big| s, \pi(s)\big) U^{\pi}(s') \qquad (4.12)$$

- By ignoring the connections between states, direct utility estimation misses opportunities for learning.
- For example, the second of the three trials given earlier reaches the state (3, 2), which has not previously been visited. The next transition reaches (3, 3), which is known from the first trial to have a high utility.
- The Bellman equation suggests immediately that (3, 2) is also likely to have a high utility, because it leads to (3, 3), but direct utility estimation learns nothing until the end of the trial.

13. ADAPTIVE DYNAMIC PROGRAMMING

- An Adaptive Dynamic Programming (or ADP) agent takes advantage of the constraints among the utilities of states by learning the transition model that connects them and solving the corresponding Markov Decision Process using a dynamic programming method.
- For a passive learning agent, this means plugging the learned transition model P(s' | s, π(s)) and the observed rewards R(s) into the Bellman equations (4.12) to calculate the utilities of the states.

- The process of learning the model itself is easy, because the environment is fully observable. This means that we have a supervised learning task where the input is a state, action pair and the output is the resulting state.
- In the simplest case, we can represent the transition model as a table of probabilities. We keep track of how often each action outcome occurs and estimate the transition probability P(s' | s, a) from the frequency with which **s'** is reached when executing action **a** in state **s**.
- For example, in the three trials, **Right** is executed three times in (1, 3) and two out of three times the resulting state is (2, 3), so P((2, 3) | (1, 3), Right) is estimated to be 2/3.

```
function PASSIVE-ADP-AGENT(percept) returns an action
  inputs: percept, a percept indicating the current state s' and reward signal r'
  persistent: π, a fixed policy
              mdp, an MDP with model P, rewards R, discount γ
              U, a table of utilities, initially empty
              N_sa, a table of frequencies for state–action pairs, initially zero
              N_s'|sa, a table of outcome frequencies given state–action pairs, initially zero
              s, a, the previous state and action, initially null

  if s' is new then U[s'] ← r'; R[s'] ← r'
  if s is not null then
      increment N_sa[s, a] and N_s'|sa[s', s, a]
      for each t such that N_s'|sa[t, s, a] is nonzero do
          P(t | s, a) ← N_s'|sa[t, s, a] / N_sa[s, a]
  U ← POLICY-EVALUATION(π, U, mdp)
  if s'.TERMINAL? then s, a ← null else s, a ← s', π[s']
  return a
```

Figure 7: A passive reinforcement learning agent based on adaptive dynamic programming. The POLICY-EVALUATION function solves the fixed-policy Bellman equations

- The full agent program for a passive ADP agent is shown in Figure 7.
- In terms of how quickly its value estimates improve, the ADP agent is limited only by its ability to learn the transition model. In this sense, it provides a standard against which to measure other reinforcement learning algorithms.
- The above algorithm is using maximum-likelihood estimation to learn the transition model; moreover, by choosing a policy based exclusively on the *estimated* model it is acting *as if* the model were correct.
- Instead, it might be a good idea to choose a policy that, while not optimal for the model estimated by maximum likelihood, works reasonably well for the whole range of models that have a reasonable chance of being the true model.

14. TEMPORAL DIFFERENCE LEARNING

- Solving the underlying MDP as in the preceding section is not the only way to bring the Bellman equations to stand on the learning problem. Another way is to use the observed transitions to adjust the utilities of the observed states so that they agree with the constraint equations.
- Consider, for example, the transition from (1, 3) to (2, 3) in the second trial. Suppose that, as a result of the first trial, the utility estimates are $U^{\pi}(1, 3) = 0.84$ and $U^{\pi}(2, 3) = 0.92$. Now, if this transition occurred all the time, we would expect the utilities to obey the equation

  $$U^{\pi}(1,3) = -0.04 + U^{\pi}(2,3)$$

(1, 1)-.04→(1, 2)-.04→(1, 3)-.04→(1, 2)-.04→(1, 3)-.04→(2, 3)-.04→(3, 3)-.04→(4, 3)+1

(1, 1)-.04→(1, 2)-.04→(1, 3)-.04→(2, 3)-.04→(3, 3)-.04→(3, 2)-.04→(3, 3)-.04→(4, 3)+1

(1, 1)-.04→(2, 1)-.04→(3, 1)-.04→(3, 2)-.04→(4, 2)-1 .

- so $U^{\pi}(1, 3)$ would be 0.88. Thus, its current estimate of 0.84 might be a little low and should be increased. More generally, when a transition occurs from state s to state s', we apply the following update to $U^{\pi}(s)$:

  $$U^{\pi}(s) \leftarrow U^{\pi}(s) + \alpha(R(s) + \gamma U^{\pi}(s') - U^{\pi}(s)) \qquad (4.13)$$

- Here, α is the **learning rate** parameter. Because this update rule uses the difference in utilities - between successive states, it is often called the **temporal-difference**, or TD, equation.
- All temporal-difference methods work by adjusting the utility estimates towards the ideal equilibrium that holds locally when the utility estimates are correct.
- This agent program for passive reinforcement learning as shown in Figure 8.

```
function PASSIVE-TD-AGENT(percept) returns an action
  inputs: percept, a percept indicating the current state s' and reward signal r'
  persistent: π, a fixed policy
              U, a table of utilities, initially empty
              N_s, a table of frequencies for states, initially zero
              s, a, r, the previous state, action, and reward, initially null

  if s' is new then U[s'] ← r'
  if s is not null then
      increment N_s[s]
      U[s] ← U[s] + α(N_s[s])(r + γ U[s'] − U[s])
  if s'.TERMINAL? then s, a, r ← null else s, a, r ← s', π[s'], r'
  return a
```

Figure 8: A passive reinforcement learning agent that learns utility estimates using temporal differences. The step-size function α(n) is chosen to ensure convergence, as described in the text

15. ACTIVE REINFORCEMENT LEARNING

- A passive learning agent has a fixed policy that determines its behavior. An active agent must decide what actions to take.
- Let us begin with the adaptive dynamic programming agent and consider how it must be modified to handle this new freedom.
- First, the agent will need to learn a complete model with outcome probabilities for all actions, rather than just the model for the fixed policy.
- The simple learning mechanism used by PASSIVE-ADP-AGENT will do just fine for this. Next, we need to take into account the fact that the agent has a choice of actions. The utilities it needs to learn are those defined by the *optimal* policy; they obey the Bellman equations.

$$U(s) = R(s) + \gamma \max_{a} \sum_{s'} P(s'|s,a)\, U(s') \qquad (4.14)$$

- These equations can be solved to obtain the utility function U using the value iteration or policy iteration algorithms.
- The final issue is what to do at each step. Having obtained a utility function U that is optimal for the learned model, the agent can extract an optimal action by one-step look-ahead to maximize the expected utility; alternatively, if it uses policy iteration, the optimal policy is already available, so it should simply execute the action the optimal policy recommends.

16. COMPARISON OF PASSIVE LEARNING AND ACTIVE LEARNING

Characteristics	Passive Learning	Active Learning
Involvement of Agent	Agents only listen and internalize.	Agent do not only sit and listen but are highly involved in the learning process.
Focus of the Learning Environment	The environment is Trainer-centered.	The environment is Agent-centered.
Role of Trainer	Trainers are masters of the subject matter.	Trainer is facilitators and co-learners.
Role of Agent	Agent is only to receive the information while being trusting and unquestioning.	The Agent's role is to question, discriminate and be critical of the information received.
Type of Thinking Involved	Passive learning facilitates convergent (coming closer together) thinking.	Active learning facilitates divergent (tending to be different or develop in different directions) thinking.

How Learning is Achieved	Learning is achieved by internalizing (incorporating) the information.	Learning is achieved by synthesizing (producing) information received.
Data	Uses a large set of pre-labeled data to train the algorithm.	Starts with a small set of labeled data and requests additional data from the user.

QUESTION BANK

1 Mark: Objective Questions

Module 1: First order logic

1. Which is an appropriate language for describing the relationships? [ ]

   (a) First-order logic (b) Propositional logic

   (c) Inductive Logic Programming (d) None of the mentioned

2. The rule of Universal Instantiation (UI for short) says that we can infer any sentence obtained by substituting a ground term (a term without variables) for the variable.

   (a) True (b) False [ ]

3. The corresponding Existential Instantiation rule: for the existential quantifier is slightly more complicated. For any sentence a, variable v, and constant symbol k that does not appear elsewhere in the knowledge base. [ ]

   (a) True (b) False

4. What among the following could the universal instantiation of _____ For all x King(x) ^ Greedy(x) => Evil(x) [ ]

   (a) King(John) ^ Greedy(John) => Evil(John)

   (b) King(y) ^ Greedy(y) => Evil(y)

   (c) King(Richar(d) ^ Greedy(Richar(d) => Evil(Richar(d)

   (d) All of the mentioned

5. Lifted inference rules require finding substitutions that make different logical expressions looks identical. [ ]

   (a) Existential Instantiation (b) Universal Instantiation

   (c) Unification (d) Modus Ponen

6. What is the process of capturing the inference process as a single inference rule?

   (a) Ponens (b) Clauses [ ]

   (c) Generalized Modus Ponens (d) Variables

7. Which process makes different logical expression looks identical? [ ]

   (a) Lifting (b) Unification (c) Inference process (d) None

8. Which algorithm takes two sentences and returns a unifier? [ ]

(a) Inference (b) Hill-climbing search

(c) Depth-first search (d) Unify algorithm

9. Which is a lifted version of modus ponens? [ ]

(a) Generalized modus ponens (b) Inference

(c) Clauses (d) None of the mentioned

10. Where did all the facts are stored to implement store and fetch function? [ ]

(a) Database (b) Knowledge base (c) Datamart (d) All of the mentioned

11. Which are more suitable normal form to be used with definite clause? [ ]

(a) Positive literal (b) Negative literal

(c) Generalized modus ponens (d) Neutral literal

12. Which algorithm will work backward from the goal to solve a problem? [ ]

(a) Forward chaining (b) Backward chaining

(c) Hill-climb algorithm (d) None of the mentioned

13. Which is mainly used for automated reasoning? [ ]

(a) Backward chaining (b) Forward chaining

(c) Logic programming (d) Parallel programming

14. What will backward chaining algorithm will return? [ ]

(a) Additional statements (b) Substitutes matching the query

(c) Logical statement (d) All of the mentioned

15. Which can be converted to inferred equivalent CNF sentence? [ ]

(a) Every sentence of propositional logic (b) Every sentence of inference

(c) Every sentence of first-order logic (d) All of the mentioned

Module 2: Learning

16. Which of the following statements is not true about the Decision tree? [ ]

(a) A Decision tree is also known as a classification tree

(b) Each element of the domain of the classification in decision tree is called a class

(c) It is a tree in which each internal node is labeled with an input feature

(d) It cannot be used in data mining applications as it only classifies but not predicts anything

17. Reinforcement learning is a _____ [ ]

(a) Prediction-based learning technique (b) Feedback-based learning technique

(c) History results-based learning technique (d) All of the mentioned

18. Which kind of data does reinforcement learning use? [ ]

(a) Labeled data (b) Unlabelled data (c) None (d) Both

19. Which of the following is the practical example of reinforcement learning? [ ]

(a) House pricing prediction (b) Market basket analysis

(c) Text classification (d) Driverless cars

20. Which of the following are applications of Reinforcement learning? [ ]

(a) Robotics (b) video games

(c) Self-driving cars (d) All of the above

21. Reinforcement learning is an area of _____. [ ]

(a) Artificial Intelligence (b) Data Analytics

(c) Machine Learning (d) None of the above

22. What is the environment in reinforcement learning? [ ]

(a) Environment is a situation that is based on the current state

(b) Environment is a situation in which an agent is present

(c) Environment is similar to feedback

(d) Environment is a situation that the agent returns as a result

23. What is an agent in reinforcement learning? [ ]

(a) Agent is the situation in which rewards are being exchanged

(b) Agent is the simple value in reinforcement learning

(c) An agent is an entity that explores the environment

(d) All the above

24. What are actions in reinforcement learning? [ ]

(a) Actions are the moves that the agent takes inside the environment

(b) Actions are the function that the environment takes

(c) Actions are the feedback that an agent provides

(d) All the above

25. What is the state of reinforcement learning? [ ]

(a) State is a situation in which an agent is present

(b) A state is the simple value of reinforcement learning

(c) A state is a result returned by the environment after an agent takes an action

(d) All the above

26. What are the Rewards of Reinforcement learning? [ ]

(a) An agent's action is evaluated based on feedback returned from the environment

(b) Environment gives value in return which is known as a reward

(c) A reward is a result returned by the environment after an agent takes an action

(d) All the above

27. What is the Policy in reinforcement learning? [ ]

(a) The agent's policy determines what environment model should be decided

(b) The agent's policy determines what action to take based on the current state

(c) The agent's policy determines what the state reward would be

(d) All the above

28. The agent's main objective is to ____the total number of rewards for good actions?

(a) Minimize (b) Maximize [ ]

(c) Maximize / Minimize (d) Null

29. Reinforcement learning is defined by the ____? [ ]

(a) Policy (b) Reward Signal

(c) Value Function (d) Model of the environment

30. Gamma (γ) in the bellman equation is known as? [ ]

(a) Value factor (b) Discount factor

(c) Environment factor (d) Learning Rate

2 Marks: Short Answer Questions

Module 1: First order logic

1. Differentiate propositional and first order inference.
2. Specify how unification process takes place.
3. Give an example for inference using forward chaining.
4. Give an example for inference using backward chaining.

Module 2: Learning

5. How an agent learning from observation? Give an example.
6. What is a Decision tree? Give an example.

7. How an agent learn using Decision trees?
8. Describe how an ADP agent will update utility values.
9. Compare active and passive Learning with respect to agent and Trainer.

5 Marks : Descriptive Questions

Module 1: First order logic

1. Explain inference rules for quantifier's propositional logic
2. Examine reduction to propositional inference with an example.
3. Describe algorithm for unification.
4. Discuss usage of first-order definite clauses in forward chaining.
5. Write simple forward-chaining algorithm.
6. What is backward-chaining? Describe algorithm for it.
7. Explain how conjunctive normal form used for first-order logic.

Module 2: Learning

8. Describe various forms of learning.
9. With an example describe representation of decision tree.
10. Explain inducing decision trees from examples.
11. Describe explanation based learning. Give an example for proof trees for the simplification problem.
12. Explain mechanism in reinforcement learning.
13. Describe Passive reinforcement learning agent.
14. Illustrate passive reinforcement learning agent based on adaptive dynamic programming.
15. Write in detail about the method for direct utility estimation.
16. Explain how the agent executes a set of trials in the environment using its policy π.
17. Explain adaptive dynamic programming.
18. Describe temporal difference learning.
19. Write the pseudo code a passive reinforcement learning agent that learns utility estimates using temporal differences.
20. Examine the approach for the implementation of active reinforcement learning.
21. Differentiate passive learning and active learning.
22. Illustrate a policy π for the 4 x 3 world and utilities for rewards of R(s) = - 0.04 in the non- terminal states and no discounting.